

A Super Mario–Style 3D Platformer

Technical Report and User Manual

Interactive Graphics Course
Sapienza University of Rome
Prof. Marco Schaerf

Academic Year 2025/2026

Abstract

This report describes a browser-playable, third-person 3D platformer built with three.js and the cannon-es physics engine, in the style of *Super Mario 3D*. The game features a fully explorable open island, a five-stage collectible quest chain, two boss encounters with dedicated obstacle courses, a rideable companion (Yoshi), a Mario-Galaxy-style walkable spherical planet with custom gravity, hand-authored procedural skeletal animation (no imported animation clips), dynamic lighting and shadows, a custom skybox shader, and a full audio and HUD/UI layer. This document covers the technical environment, the third-party assets and libraries used, the architecture of the codebase, and a walkthrough of every implemented interaction, in line with the course’s project requirements.

Contents

1	Introduction	2
2	Environment and Third-Party Libraries	2
2.1	Rendering and physics environment	2
2.2	Libraries, tools and assets not developed by the team	2
3	Scene Composition and Hierarchical Models	3
3.1	Level architecture	3
3.2	Hierarchical models	3
4	Animation System	4
4.1	Rig-agnostic pose synthesis	4
4.2	Runtime playback	4
5	Lighting, Materials and Shaders	5
5.1	Lighting	5
5.2	Textures	5
5.3	Custom skybox shader	5
6	Physics and the Planetary Gravity System	5
6.1	Rigid-body simulation	5
6.2	Mario-Galaxy-style spherical gravity	6
7	Camera System	6
8	Gameplay Systems and User Interaction	6
8.1	Controls	6
8.2	Core movement and mounting	6
8.3	Enemies and boss fights	7

8.4	Warp stars and the sky planet	7
8.5	Quest system and dynamic HUD	7
8.6	HUD and menus	7
9	Audio	8
10	Ending Sequence	8
11	Conclusion	8
1	Introduction	

The project is a single-player, third-person 3D platform game rendered entirely in the browser via WebGL. The player explores an open island, collects coins and Power Stars, completes a linear quest given by an NPC (Toad), fights two bosses (Kamek and Bowser) in separate obstacle-course arenas, can mount and ride a companion character (Yoshi), and can travel to a small walkable “sky planet” through a Mario-Galaxy-style warp mechanic before finally reaching Princess Peach for a scripted ending sequence.

The game is implemented as a client-side single-page application: a single `index.html` entry point loads a module-based JavaScript codebase (organized under `js/`) that sets up the `three.js` scene, the `cannon-es` physics world, the asset pipeline, and the game loop. No server-side component or build step is required – the project runs directly from static files and is deployed via GitHub Pages, as requested by the project guidelines.

2 Environment and Third-Party Libraries

2.1 Rendering and physics environment

The project is built on top of **three.js** (r160), used as the WebGL abstraction layer for scene graph management, cameras, materials, lighting, shadow mapping, GLTF loading and skeletal animation primitives. Physics simulation (rigid bodies, collision detection, contact resolution) is delegated to **cannon-es** (v0.20.0), a maintained, ES-module fork of `cannon.js`, chosen among the physics engines suggested in the project guidelines for its lightweight footprint and straightforward integration with `three.js`. Both libraries are imported directly as ES modules from public CDNs (unpkg for `three.js`, jsDelivr for `cannon-es`), so no local bundling step is required.

2.2 Libraries, tools and assets not developed by the team

Table 1 lists every external dependency, tool, and asset used in the project that was not created by the team. Everything else described in this report – the game logic, the procedural animation system, the physics tuning, the level format, the custom shaders, and the level-design content – was written by the team.

Notably, per the project constraints, **no animation clips were imported**: every character animation (idle, walk, run, jump, fall, boss wind-up poses, Yoshi’s leg-only walk cycle) is synthesized procedurally by the team’s own code from quaternion keyframes, described in Section 4. Two tileable arena textures (a lava-brick material for Bowser’s arena and a purple “magic brick” material for Kamek’s) were also generated procedurally by the team with a small offline Python/PIL script rather than downloaded, and are treated as team-authored content.

Dependency	Purpose
three.js (r160)	Core WebGL rendering engine: scene graph, cameras, materials, lights, shadow maps, <code>GLTFLoader</code> , <code>DRACOLoader</code> , <code>SkeletonUtils</code> , <code>RoomEnvironment</code> .
cannon-es (0.20.0)	Rigid-body physics engine: dynamic/static bodies, sphere/box colliders, contact materials, collision filter groups.
cannon-es-debugger	Optional wireframe visualization of every physics collider (toggled with the F3 key), used during development to align visual meshes with their colliders.
stats.js	Real-time FPS/frame-time performance overlay, used for development profiling.
DRACO decoder (Google, hosted on <code>gstatic.com</code>)	Decompression of Draco-compressed glTF mesh geometry.
Google Fonts (“Press Start 2P”)	Retro pixel-style typography used throughout the HUD and menus.
glTF 3D models	Mario, Luigi, Yoshi, Goomba, Kamek, Bowser, Princess Peach and her castle, plus every static prop (coins, stars, mushrooms, question blocks, trees, grass, the Toad house, Yoshi’s egg, warp stars, an Oak-Sign prop). All were downloaded/pre-modeled, not authored by the team; several were rigged with a Mixamo-style humanoid skeleton so the team’s own procedural animation system could drive them.
Photographic/painted textures	Sky-box panorama, ground grass texture, and the sky planet’s color/normal/roughness map set.
GitHub Pages	Static hosting for the playable build, as required by the project guidelines.

Table 1: Third-party libraries, tools and assets used by the project.

3 Scene Composition and Hierarchical Models

3.1 Level architecture

The game world is data-driven: the main island, and each of the two boss obstacle courses, are described by JSON files (`level1.json`, `kamek_zone.json`, `bowser_zone.json`, plus a small `peach_castle.json` for the ending area) listing platform geometry, buildings, NPCs, hills, and special points of interest (spawn point, Yoshi’s egg location, boss zone entrances). A dedicated `LevelLoader` module parses this data and instantiates the corresponding three.js/cannon-es objects, so new content can be added without touching game logic code.

On top of the JSON-driven skeleton of the level, a `Decorations` module procedurally scatters trees, flowers, rocks, bushes and coin trails across the island using a jittered grid/randomized placement algorithm, with explicit rectangular and circular exclusion zones so that props never spawn inside buildings, hill staircases, or on top of interactive elements such as warp stars and their signage. Grass is rendered as a single `InstancedMesh` covering the whole island for performance, and rocks/bushes are each merged into one draw call via `BufferGeometryUtils.mergeGeometries`, keeping the scene’s draw-call count low despite dozens of individually-varied instances.

3.2 Hierarchical models

The project satisfies the “at least one complex hierarchical model” requirement in two complementary ways:

- **Skeletal character rigs.** Every playable character, Yoshi, and every enemy that has a walk cycle (currently Bowser, in addition to the two playable characters and Yoshi) is a

fully rigged, multi-bone hierarchy (hips → spine → chest, and left/right shoulder → upper arm → forearm → hand, hip → thigh → shin → foot → toe chains). The team’s own animation system (Section 4) drives this hierarchy directly, rotating each bone relative to its parent to produce full-body poses – this is precisely the kind of hierarchical, parent-relative transform chain the requirement asks for, and it is exercised on every frame of gameplay.

- **Composite level structures.** Buildings (the Mario/Toad houses), the three-platform “HillClimb” staircases, the two boss arenas, and the walkable sky planet are each built as a group of individually-collided sub-objects parented under a single three.js **Group**, so they can be positioned, decorated and given colliders as one logical unit while still resolving physics per sub-platform.

4 Animation System

Because imported animation clips are disallowed, the team implemented a from-scratch procedural animation pipeline (`clipFactory.js`, `BoneMap.js`, `AnimationController.js`) that works uniformly across every rigged model, regardless of the humanoid rig’s own bone-naming convention or local axis orientation.

4.1 Rig-agnostic pose synthesis

`BoneMap` inspects a loaded model’s skeleton and maps semantic roles (*hips*, *spine*, *upperArmL*, *thighR*, *footL*, ...) to the actual bone nodes found in the GLTF scene graph, tolerating naming variations across different rigs. At load time, `clipFactory.characterBasis()` derives an orthonormal *up/left/forward* reference frame directly from the character’s bind pose (up from hips to spine; forward from the toe-to-heel vector, with a shoulder-vector fallback for rigs without toe bones), so every animation pose is expressed in *semantic* terms (“swing this thigh forward by X degrees”) rather than in raw, rig-specific local-axis rotations. This is what lets the same hand-authored pose parameters (grouped in a single tunable **POSE** table) drive Mario, Luigi, Yoshi and Bowser identically despite their rigs having been authored independently.

Each pose is built by computing, for every limb, a target world-space direction from the character’s own basis, then solving for the local bone quaternion that rotates the limb’s rest orientation to point that way – entirely independent of how the source rig’s local axes happen to be oriented. From these poses the factory builds full **THREE.AnimationClip** objects (idle, walk, run, jump, fall) with quaternion keyframe tracks per bone.

4.2 Runtime playback

`AnimationController` wraps a **THREE.AnimationMixer** per character and exposes a small state machine driven every frame by the character’s own physical motion (horizontal speed, vertical velocity, grounded flag): idle/walk/run/jump/fall states are cross-faded into one another (0.18 s blend), and the walk/run cycles’ playback rate is continuously rescaled to the character’s actual speed so the step rate never visually “skates”. A separate `lock()/unlock()` API lets game logic override the motion-driven state machine for a scripted held pose – used for Bowser’s fire-breath wind-up animation, which must hold even though he is standing still (which the state machine would otherwise read as “idle”). Yoshi uses a reduced clip set (`buildYoshiClips`) that only animates his legs, leaving the rest of his rig static, matching his lower-detail rig.

5 Lighting, Materials and Shaders

5.1 Lighting

The scene uses a two-light setup: a warm-tinted `AmbientLight` for base illumination, and a single shadow-casting `DirectionalLight` that follows the player every frame. Its orthographic shadow camera is deliberately kept tight around the player (a ± 30 -unit box at 2048^2 shadow-map resolution) rather than covering the whole island, trading total shadow coverage for much higher local shadow sharpness – satisfying the “at least one light” requirement while keeping shadow quality high near the action. `PCFSoftShadowMap` filtering is enabled for soft shadow edges, and the renderer uses linear tone mapping with a hand-tuned exposure value together with a PBR-neutral environment map (three.js’ `RoomEnvironment`, baked via `PMREMGGenerator`) so that metallic glTF materials (coins, stars) have something to reflect instead of rendering flat black.

5.2 Textures

Multiple texture types are used, matching the “textures of different kinds” requirement:

- **Color/albedo maps** on essentially every glTF model and on the ground.
- **Normal and roughness maps** on the walkable sky planet, giving its surface real relief and controlled specular response under the directional light.
- **Procedurally generated tileable textures**: two brick materials (lava-stone for Bowser’s arena, purple “magic brick” for Kamek’s) were synthesized offline with a Python/PIL script rather than downloaded, so their exact color palette and tiling could be tuned to the level design.
- **Decal textures**: small alpha-blended logo planes overlaid on signpost props to mark the entrance of each boss zone.

5.3 Custom skybox shader

The sky is a large inward-facing sphere (radius 1000, rendered first with `renderOrder = -1` and `depthWrite` disabled to avoid z-fighting) using a hand-written GLSL `ShaderMaterial`. The fragment shader samples the panoramic sky texture, derives per-pixel luminance, and blends a darkened base sky with a contrast-boosted “bright clouds” pass keyed by that luminance, giving a moodier sky than the source texture alone while keeping clouds well-defined. The sphere is re-centered on the camera every frame so it always reads as an infinite backdrop.

6 Physics and the Planetary Gravity System

6.1 Rigid-body simulation

All dynamic actors (player, enemies, Yoshi) are represented in cannon-es as fixed-rotation dynamic spherical bodies, synced to their three.js mesh every frame; static level geometry (ground, platforms, rocks, the sky planet) uses static bodies. A shared `ContactMaterial` with near-zero friction and no restitution is used everywhere, since horizontal movement is driven directly by the character controller rather than by friction. Collision *filter groups* (`GROUND`, `PLAYER`, `ENEMY`, `YOSHI`) are used to deliberately *exclude* certain body pairs from producing a real physics response – e.g. the player and an enemy never physically collide with each other, since every player/enemy interaction (stomp, side damage) is already handled by scripted game logic; letting cannon-es’ own solver also push the two overlapping spheres apart on top of that produced a visible double-bounce bug that this filtering removes at the source.

6.2 Mario-Galaxy-style spherical gravity

On top of the flat world gravity, the project layers an additive, opt-in **GravityField** system: a spherical gravity well with a center, a radius, an influence radius and a strength, registered against the physics engine. Any body that opts in (currently only the player) is, while within a field's influence radius, pulled radially toward that field's center instead of falling under the world's flat “down” gravity – implemented by canceling the flat-gravity force cannon-es is about to apply and substituting a radial pull toward the field's center, computed fresh every physics step.

This is what powers the walkable “red planet”: once the player enters its gravity field, they fall toward its surface from any direction and can walk around it in any direction, including upside-down relative to the rest of the world. Movement direction, character orientation and the camera all had to be reworked to operate in the planet's local tangent frame rather than in absolute world axes: a remembered forward/camera direction is *parallel-transported* onto the current tangent plane every frame (projecting out its component along the local “up” and renormalizing) instead of being recomputed from scratch, which is what keeps steering and the camera continuous as the player walks over the curved surface – a naive per-frame recomputation produces a dead zone and abrupt direction flips near the poles of the sphere.

7 Camera System

The project implements two distinct camera behaviors, chosen automatically every frame based on whether the player is currently inside a gravity field:

- **Flat/orbital mode** (everywhere on the main island and inside the boss arenas): a classic third-person orbit camera around the character, manually steerable with the I/J/K/L keys, with a smooth auto-follow that gently re-aligns behind the character while they are moving forward and not being manually steered.
- **Rigid planetary mode** (on the walkable sky planet): manual orbit is disabled; the camera is locked rigidly over the character's shoulder, with its own “up” vector re-aimed to the planet's local up every frame (critical, since `THREE.Camera.lookAt` resolves roll from `camera.up` – without this the view would tip over as the character walks around the curve). The camera's own trailing direction is likewise parallel-transported and steered toward the character's facing with a bounded angular step (rather than a linear vector interpolation, which degenerates to zero during near-180° turns) to avoid fighting with the character's own steering.

A third, fully scripted fixed cinematic shot is used once, for the ending dialogue with Princess Peach: the camera holds a framing in front of her, offset so she reads on the right third of the frame, for the duration of the scripted dialogue.

8 Gameplay Systems and User Interaction

8.1 Controls

8.2 Core movement and mounting

The player character controller supports walking, running (sprint), jumping, and full free camera-relative steering, all layered on top of the cannon-es rigid body described in Section 5. A second, independent character – Yoshi – can be hatched from an egg found on the island and then mounted; while ridden, Yoshi's own physics body is kept locked to the player's position and the player's jump is boosted, letting the player reach an otherwise unreachable elevated collectible

Input	Action
W / A / S / D	Move (forward / left / back / right)
Shift	Sprint
Space	Jump
I / J / K / L	Free camera orbit (pitch/yaw) – disabled while on the sky planet
E	Interact: talk to an NPC, mount/dismount Yoshi, advance dialogue
Esc / Pause button	Pause menu (volume sliders, mute, controls reference, restart)
F3	Toggle physics collider wireframe overlay (debug)

Table 2: Player controls.

star. Yoshi can be dismounted at will, and is returned to his egg if lost (e.g. by falling into the void while ridden), so the mechanic is always recoverable.

8.3 Enemies and boss fights

A shared **Enemy** base class implements a simple detection-range/chase AI and a generic “stomp to damage” interaction, used by three enemy types: Goomba (a basic, one-hit patrol enemy on the main island) and two multi-hit bosses – Kamek and Bowser – each guarding the end of its own dedicated obstacle-course level (`kamek_zone.json` / `bowser_zone.json`), complete with lava hazards, a wind-up ranged attack, and a live boss health bar shown while the player is inside the arena. Defeating a boss drops both a collectible Power Star and a warp star that returns the player to the main island.

8.4 Warp stars and the sky planet

Several “launch star” props scattered around the island act as one-way teleporters: the game checks player proximity to each star every frame and, on contact, repositions the player’s physics body to that star’s configured destination (the sky planet, the level spawn point, or the entrance of either boss zone). A dedicated yellow warp star sends the player to the walkable “red” sky planet described in Section 5.2; a warning banner is shown if the player approaches a warp star while still mounted on Yoshi, since warp destinations are intentionally inert while ridden.

8.5 Quest system and dynamic HUD

A **QuestManager** module drives a top-right, always-visible objective panel guiding the player through five sequential collectible “Power Star” phases: reaching the sky planet, hatching and riding Yoshi to an elevated star, and a three-step linear quest given by an NPC (Toad) that first sends the player to defeat Kamek, then to collect 25 coins, then to defeat Bowser – each step requiring the player to report back to Toad in person (via the same **E**-to-interact prompt used everywhere else) before the next step unlocks. Toad’s own lines are shown through a reusable speech-bubble dialogue widget (also used for Peach’s ending scene), while ambient notices (a boss defeated, a star picked up) surface as brief toasts. The two independent side-objectives (the planet and Yoshi’s star) can be completed in any order; only Toad’s own three-step chain is strictly sequential.

8.6 HUD and menus

The heads-up display shows the player’s star and coin counts, remaining lives, an optional boss health bar, the current quest objective, and contextual “Press E to ...” prompts. A pause menu (opened with **Esc** or a dedicated button) exposes independent music and SFX volume sliders, a master mute toggle, a full controls reference, and a restart option; a separate two-step start

flow lets the player choose between Mario and Luigi and enter a custom name before spawning, which is then used throughout the run (including by name in Peach’s final dialogue).

9 Audio

An `AudioManager` module owns two independently volumed channels (background music and sound effects), a master mute toggle, and a small per-context “voice” system: sound effects that have a Yoshi-specific variant (jump, fall) automatically switch to it while the player is riding him, and back to the default character voice on dismount. Background music switches automatically between the overworld theme and each boss zone’s own battle track as the player crosses into and out of that zone, with a one-shot greeting line played once per visit to a boss’s zone.

10 Ending Sequence

Once every required star has been collected, a win screen offers the player a final walk to Princess Peach’s castle. Reaching her triggers a short, fully scripted cutscene: player input is completely suspended, the camera switches to the fixed cinematic shot described in Section 6, and a short multi-line dialogue (using the hero’s chosen name) plays out before a final “Thank You” screen is shown. A dedicated game state guards this final screen so that, unlike the mid-dialogue freeze (which only halts the character), the whole game loop – camera, interaction prompts, boss-zone music switching – is inert while it is on screen.

11 Conclusion

The project combines the elements required by the course specification – a hierarchical, skeletally-animated model driven entirely by hand-written procedural animation; multiple light and texture types, including a custom GLSL skybox shader and procedurally generated materials; a rich set of player-facing interactions; and a physics-driven, non-trivial gameplay mechanic in the form of the planetary gravity system – into a single cohesive, browser-playable game. Beyond the minimum requirements, particular engineering effort went into making the animation system rig-agnostic, into the parallel-transport technique that keeps movement and the camera stable on a curved, gravity-wrapped surface, and into a quest/HUD layer that guides the player through the game’s content without hard-gating any of it.